Recursión, pilas, colas y diccionarios: soluciones

Programación: UVa 839, UVa 12100 y UVa 156

Carlos A Delgado S

Pontificia Universidad Javeriana Cali
Sesiones de refuerzo de algoritmia

Contenido

1 Recursión sobre un árbol

2 Pilas y colas

3 Diccionarios y conjuntos

Contenido

1 Recursión sobre un árbol
- Solución: UVa 839

2 Pilas y colas

3 Diccionarios y conjuntos

La idea

Una función por barra

`movil` lee una barra. Por cada extremo con peso 0 se llama a sí misma, y esa llamada lee el submóvil completo y devuelve su peso y si está en equilibrio. Con los dos pesos de verdad compara $W_l D_l$ con $W_r D_r$. Un extremo con objeto es el caso base, porque su peso ya viene en la línea. El árbol de llamadas es el móvil.

Python: la función `movil`

```
def movil(datos, pos):
    # lee la barra que empieza en datos[pos] y todo lo que cuelga de ella;
    # devuelve el peso, si está en equilibrio y dónde sigue la entrada
    wl = int(datos[pos])
    dl = int(datos[pos + 1])
    wr = int(datos[pos + 2])
    dr = int(datos[pos + 3])
    pos = pos + 4
    equilibrio = True
    if wl == 0:
        wl, eq_izq, pos = movil(datos, pos)
        equilibrio = equilibrio and eq_izq
    if wr == 0:
        wr, eq_der, pos = movil(datos, pos)
        equilibrio = equilibrio and eq_der
    if wl * dl != wr * dr:
        equilibrio = False
    return wl + wr, equilibrio, pos
```

Python: los casos

```
import sys
def main():
    sys.setrecursionlimit(10000)
    datos = sys.stdin.read().split()
    casos = int(datos[0])
    pos = 1
    for c in range(casos):
        peso, equilibrio, pos = movil(datos, pos)
        if c > 0:
            print("")
        if equilibrio:
            print("YES")
        else:
            print("NO")


main()
```

C++: la función `movil`

```cpp
// lee la barra y todo lo que cuelga de ella; devuelve el peso y deja
// equilibrio en false si alguna barra del submóvil no está en equilibrio
long long movil(bool& equilibrio) {
    long long wl, dl, wr, dr;
    scanf("%lld %lld %lld %lld", &wl, &dl, &wr, &dr);
    if (wl == 0) {
        wl = movil(equilibrio);
    }
    if (wr == 0) {
        wr = movil(equilibrio);
    }
    if (wl * dl != wr * dr) {
        equilibrio = false;
    }
    return wl + wr;
}
```

Un solo valor de vuelta

Devuelve el peso, y `equilibrio` va por referencia: la barra que falla lo apaga y ninguna lo vuelve a encender.

C++: los casos

```cpp
#include <cstdio>
int main() {
    int casos;
    scanf("%d", &casos);
    for (int c = 0; c < casos; c++) {
        bool equilibrio = true;
        movil(equilibrio);
        if (c > 0) {
            printf("\n");
        }
        if (equilibrio) {
            printf("YES\n");
        } else {
            printf("NO\n");
        }
    }
    return 0;
}
```

Trampa: dejar de leer cuando ya es `NO`

Caso	1	2	3	4	5	6
Esperado	NO	YES	NO	NO	NO	YES
Sin leer el resto	NO	YES	YES	YES	NO	YES

Las líneas quedan sin consumir

Si la función no lee el submóvil derecho cuando el izquierdo ya falló, sus líneas se leen como el caso siguiente. La tabla compara las dos versiones sobre los seis casos de `2.in`.

Otras trampas

Lectura, salida y tamaño

- Se lee por palabras, y las líneas en blanco no cuentan.
- La línea en blanco de la salida va entre dos casos, no después del último.
- El enunciado no acota los pesos, y la comparación multiplica un peso acumulado por una distancia: en C++ van en `long long`.

El costo

Con b barras

- El tamaño es b, que el enunciado no acota.
- Hay una llamada por barra, cada una con trabajo fijo: $\Theta(b)$ por caso, igual que leer la entrada.
- La pila llega a la altura del móvil, hasta b marcos en una cadena. Por eso Python sube su límite de 1000 niveles con `sys.setrecursionlimit`.

Contenido

1 Recursión sobre un árbol

2 Pilas y colas
- Solución: UVa 12100

3 Diccionarios y conjuntos

La idea

Simular con una cola de pares

Cada elemento es la prioridad y si es el trabajo propio. Se saca el frente y se miran los contadores de las prioridades mayores. Si alguno es positivo, el par vuelve al final. Si no, su contador baja, pasa un minuto, y la simulación termina cuando el par impreso es el marcado.

Python: hay_mayor_prioridad

```python
def hay_mayor_prioridad(cuenta, p):
    hay = False
    for q in range(p + 1, 10):
        if cuenta[q] > 0:
            hay = True
    return hay
```

Una pregunta, una función

Revisa los contadores de las prioridades por encima de p, sin tocar la cola: a lo más ocho.

Python: minutos_hasta_imprimir

```python
# cola de pares (prioridad, es_mio); cuenta[p] dice cuántos trabajos
# de prioridad p siguen en la cola
def minutos_hasta_imprimir(prioridades, m):
    cola = deque()
    cuenta = [0] * 10
    for i in range(len(prioridades)):
        cola.append((prioridades[i], i == m))
        cuenta[prioridades[i]] = cuenta[prioridades[i]] + 1
    minutos = 0
    impreso = False
    while not impreso:
        p, es_mio = cola.popleft()
        if hay_mayor_prioridad(cuenta, p):
            cola.append((p, es_mio))
        else:
            cuenta[p] = cuenta[p] - 1
            minutos = minutos + 1
            impreso = es_mio
    return minutos
```

Python: la lectura

```
import sys
from collections import deque
```

```
def main():
    datos = sys.stdin.read().split()
    casos = int(datos[0])
    pos = 1
    for c in range(casos):
        n = int(datos[pos])
        m = int(datos[pos + 1])
        pos = pos + 2
        prioridades = []
        for i in range(n):
            prioridades.append(int(datos[pos + i]))
        pos = pos + n
        print(minutos_hasta_imprimir(prioridades, m))


main()
```

C++: hayMayorPrioridad

```
bool hayMayorPrioridad(const vector<int>& cuenta, int p) {
    bool hay = false;
    for (int q = p + 1; q < 10; q++) {
        if (cuenta[q] > 0) {
            hay = true;
        }
    }
    return hay;
}
```

Referencia constante

cuenta llega como `const vector<int>&`: la función lee el vector sin copiarlo y sin poder cambiarlo.

C++: minutosHastaImprimir

```cpp
int minutosHastaImprimir(const vector<int>& prioridades, int m) {
    queue<pair<int, bool> > cola;
    vector<int> cuenta(10, 0);
    for (int i = 0; i < (int) prioridades.size(); i++) {
        cola.push(make_pair(prioridades[i], i == m));
        cuenta[prioridades[i]] = cuenta[prioridades[i]] + 1;
    }
    int minutos = 0;
    bool impreso = false;
    while (!impreso) {
        pair<int, bool> trabajo = cola.front();
        cola.pop();
        if (hayMayorPrioridad(cuenta, trabajo.first)) {
            cola.push(trabajo);
        } else {
            cuenta[trabajo.first] = cuenta[trabajo.first] - 1;
            minutos = minutos + 1;
            impreso = trabajo.second;
        }
    }
    return minutos;
}
```

C++: la lectura

```cpp
#include <cstdio>
#include <queue>
#include <utility>
#include <vector>

using namespace std;
```

```cpp
int main() {
    int casos;
    scanf("%d", &casos);
    for (int c = 0; c < casos; c++) {
        int n, m;
        scanf("%d %d", &n, &m);
        vector<int> prioridades(n);
        for (int i = 0; i < n; i++) {
            scanf("%d", &prioridades[i]);
        }
        printf("%d\n", minutosHastaImprimir(prioridades, m));
    }
    return 0;
}
```

Trampas

Lo que falla

- Comparar la prioridad en lugar de la marca: el tercer caso da 2 en lugar de 5.
- Cobrar un minuto por rotar: la muestra da 1, 7 y 7 en lugar de 1, 2 y 5.
- Bajar el contador al rotar: la muestra da 1, 4 y 5, porque el trabajo sigue en la cola.

El costo

Con $n \leq 100$ trabajos y hasta 100 casos

- Cada impresión deja a lo más n rotaciones por delante: a lo más n^2 pasos.
- Cada paso revisa a lo más ocho contadores: $O(n^2)$ por caso, 10^4 pasos con $n = 100$ y 10^6 con los 100 casos.
- Con una lista de Python sacando del frente, cada paso paga $\Theta(n)$ y el total sube a $O(n^3)$.

Contenido

1 Recursión sobre un árbol

2 Pilas y colas

3 Diccionarios y conjuntos
- Solución: UVa 156

La idea

Contar por clave

Se lee palabra por palabra hasta el #, se guarda cada palabra y se suma uno a la cuenta de su clave: sus letras en minúscula y ordenadas. Al final se ordenan las palabras originales y se imprimen las que tienen cuenta 1 en su clave.

Python: la clave

```
import sys


def clave(palabra):
    # las letras en minúscula y ordenadas: dos anagramas dan la misma clave
    letras = sorted(palabra.lower())
    resultado = ""
    for c in letras:
        resultado = resultado + c
    return resultado
```

Python: el programa

```
def main():
    datos = sys.stdin.read().split()
    palabras = []
    cuenta = {}
    i = 0
    while i < len(datos) and datos[i] != "#":
        k = clave(datos[i])
        if k in cuenta:
            cuenta[k] = cuenta[k] + 1
        else:
            cuenta[k] = 1
        palabras.append(datos[i])
        i = i + 1
    for p in sorted(palabras):
        if cuenta[clave(p)] == 1:
            print(p)


main()
```

C++: la clave

```cpp
#include <algorithm>
#include <cctype>
#include <cstdio>
#include <map>
#include <string>
#include <vector>

using namespace std;

// las letras en minúscula y ordenadas: dos anagramas dan la misma clave
string clave(const string& palabra) {
    string k = palabra;
    for (int i = 0; i < (int) k.size(); i++) {
        k[i] = tolower(k[i]);
    }
    sort(k.begin(), k.end());
    return k;
}
```

C++: el programa

```
int main() {
    char leida[100];
    vector<string> palabras;
    map<string, int> cuenta;
    int ok = scanf("%s", leida);
    while (ok == 1 && leida[0] != '#') {
        string p = leida;
        cuenta[clave(p)] = cuenta[clave(p)] + 1;
        palabras.push_back(p);
        ok = scanf("%s", leida);
    }
    sort(palabras.begin(), palabras.end());
    for (int i = 0; i < (int) palabras.size(); i++) {
        if (cuenta[clave(palabras[i])] == 1) {
            printf("%s\n", palabras[i].c_str());
        }
    }
    return 0;
}
```

Trampas

Lo que falla

- La clave sin minúscula da 16 palabras en lugar de 7.
- Ordenar sin distinguir mayúsculas empieza por `derail`, `Disk`, y lo correcto empieza por `Disk`, `NotE`.
- Leer por líneas falla: una línea trae varias palabras y puede empezar con espacios.

El costo

Con $n \leq 1000$ palabras de $L \leq 20$ letras

- Las claves, ordenando las letras: $O(n\,L \log L)$, menos de 10^5 pasos.
- La salida: $O(n \log n)$ comparaciones, unas 10^4, de hasta L letras cada una.
- Comparar cada palabra con las demás pagaría $\Theta(n^2)$ comparaciones de claves: unos 500 000 pares.